

PointSAT: usable workflows, automatic compaction, and measured solver improvements

Technical experiment report

6 September 2026

Abstract

A compact engineering and experimental record of the PointSAT work. It separates software improvements, exact constructions, fixed-work speed, coordinate-size reductions and matched search outcomes. The emphasis is reproducibility and generality across point sets rather than fitting one published grid.

Contents

1	Scope and principal outcomes	2
2	A usable, compact workflow	2
2.1	Storage and recovery	2
3	Automatic integer-coordinate optimization	3
3.1	Matched-input exploratory comparison	3
3.2	Frozen generalization and same-mode ablation	4
4	PointSAT and Localizer changes	5
4.1	Established speed evidence	5
4.2	Additional micro-optimizations tested, not promoted	5
5	Matched feedback-on/off experiments	6
6	Earlier geometry and the exact 19-point result	7
7	Validation, limitations, and next work	7
8	Source and artifact inventory	7

1 Scope and principal outcomes

This report covers the direct-geometric work and PointSAT/Localizer improvements of 5 September, followed by the usability, coordinate-optimization and matched experiments of 6 September 2026. The supplied `Happy_ending.pdf` is *Toward Satisfiability Modulo Realizability*. Its Figure 1 and Section 5.2 specify a 672×566 integer witness with no empty convex hexagon and no convex heptagon. A 64×78 result mentioned from another conversation was not available as a fixture; it was a target, not a claimed reproduced result.

The implementation now provides a compact command-line workflow, a self-contained optimized Localizer source tree, automated multi-solution compaction, explicit hull-layer preservation modes, and independently checked outputs. The paper example reached 101×78 with frozen experimental v2 in 600 seconds, without target-grid options. No claim of global coordinate optimality is made. The user's final priority was generality across inputs rather than tuning to that example; subsequent tests use independently discovered types and all four families.

Earlier SAT-guided work solved the supplied 19-point problem with exact Euclidean threefold symmetry and produced at least 15 distinct 23-point order types. The new matched experiments also found exactly verified 29-point realizations. These are verified constructions, not claims of literature-wide novelty or new bounds for every family.

2 A usable, compact workflow

```
make
python -m pointsat doctor
python -m pointsat run --problem mixed23 --samples 20 \
    --workers 2 --seconds 600 --out runs/demo
python -m pointsat status runs/demo
python -m pointsat optimize runs/demo --seconds 60 --mode layers
python -m pointsat solutions runs/demo --csv
python -m pointsat export runs/demo --mode layers --out exports/demo --svg
```

The CLI accepts the four paper presets and advanced legacy settings. It detects existing external SAT tools and uses `localizer/build/localizer` by default. Search and automatic per-solution optimization budgets are separate and explicit. The old `PointSAT.py SETTINGS.json` command remains compatible; the modern `run -settings` command selects compact output instead. The presets are `mixed23` (no 6-hole or convex 7-gon), `holes29` (no 6-hole), `gons32` (no convex 7-gon), and `caps26` (no convex 7-gon or 5-cap in the actual x direction).

2.1 Storage and recovery

One SQLite database stores compressed JSON events, exact coordinate arrays, configuration/provenance metadata, and content-addressed original artifacts. Only the coordinator writes. Temporary WAL files permit inspection while running. Completed jobs reclaim native input/output files after the last queued or active reference disappears. A normally completed run leaves one database, not one directory of files per attempt. Explicit export can produce points, SVG diagrams, events, and one ZIP of original artifacts with an event/path index.

An event, its artifacts and its accepted solution commit atomically. Interrupted transactions retain their bounded temporary workspace; `recover` RUN independently checks saved coordinates and deduplicates repeated recovery. Exceptional output is not deleted merely because a database write was interrupted. Existing research directories are never silently removed or repacked. Recovery does not pretend to resume the identical stochastic search state.

Storage replay	Legacy	SQLite
Final files	1,804	1
Logical MiB	12.50	3.09
Allocated MiB	18.42	3.09
Peak scratch files	1,800	6
Median wall seconds	1.743	1.532
Median CPU seconds	1.736	1.440

This deterministic replay used 200 representative SAT records and 400 native-attempt records, three repetitions and alternating storage order. Semantic payloads matched after ordering by job ID. It performs no SAT or native search: its timings isolate storage/coordinator overhead, not end-to-end solution speed. Allocated disk usage decreased by 83.2%. The first draft hash check compared different event orders and failed; the corrected replay compares the same semantic records by ID. Raw draft and corrected results are retained.

3 Automatic integer-coordinate optimization

The tool discovers valid realizations in a collection or in a legacy run’s `realizations/` directory. New successful searches automatically feed their saved solutions to the optimizer; no manually paired orientation file is needed. Input decimals are interpreted exactly and positive axis scaling clears denominators. Bounded rounding is used only if all triple signs are preserved; coordinate-dependent cap validity is separately rechecked.

The standard native C++ portfolio combines exact feasible-line moves, valid order-type boundary crossings, affine/projective proposals with integer rounding and exact repair. Experimental v2 adds minimum-triangle-area centering and triangular-hull barycentric normalization. The latter is a general preconditioner, not a hard-coded transformation of the paper coordinates. Floating point proposes moves only; bounded signed-128-bit predicates guard acceptance. Optional actual-geometry repair and target-grid proposals are not enabled by default. The best valid incumbent is saved atomically, at most once per second and on termination, rather than dumping every improvement.

layers (CLI default) Preserve the sequence of hull-layer sizes; labeled orientation signs and membership of individual layers may change.

order-type Preserve every labeled orientation sign.

free Permit any configuration satisfying the same geometric family.

Preservation refers to the original imported solution, not an earlier free optimization that may have changed its layers. Listing and export accept a matching preservation filter; without one, they select the smallest saved box overall and report its actual layer sizes. The objective is bounding-box area, with side-length tie-breaking in the native engine. Width and height are coordinate spans: a $w \times h$ box has $(w + 1)(h + 1)$ integer grid sites. Every accepted optimized result has a separate exact geometric audit, before/after grid dimensions and layers, orientation changes, seed, budget, and binary hash. A smaller result need not satisfy the source CNF’s label-symmetry-breaking clauses after its order type changes.

3.1 Matched-input exploratory comparison

Six inputs were run for 120 native wall seconds each, one seed, at most three single-core jobs concurrently. The old compactor preserves the complete order type; the initial new optimizer v1 used its more permissive default. Therefore this measures the benefit of a broader compaction strategy, not a pure speed ablation under identical preservation constraints. All twelve outputs passed independent exact validation.

Input	Old box	New v1 box
paper23	671×565	253×253
archive-sample7	27,234×36,540	686×1,908
archive-sample2	108,059×94,525	238×315
sat-guided-case33	256,366×125,237	4,080×1,485
feedback-sample3	252,429×484,870	147×95
control-margin-sample6	41,454×26,999	1,089×2,135

Area improved on all six exploratory examples. Five independently discovered inputs represent two original layer-size classes, not five independent layer classes. One free compaction changed layers from (3, 4, 4, 6, 5, 1) to (3, 3, 4, 6, 4, 3); that result must not be presented as layer-preserving. Frozen v2 additionally produced a 101×121 example from an independently discovered input while retaining its layers. Other inputs remain substantially larger under the same budget, so performance is not uniform.

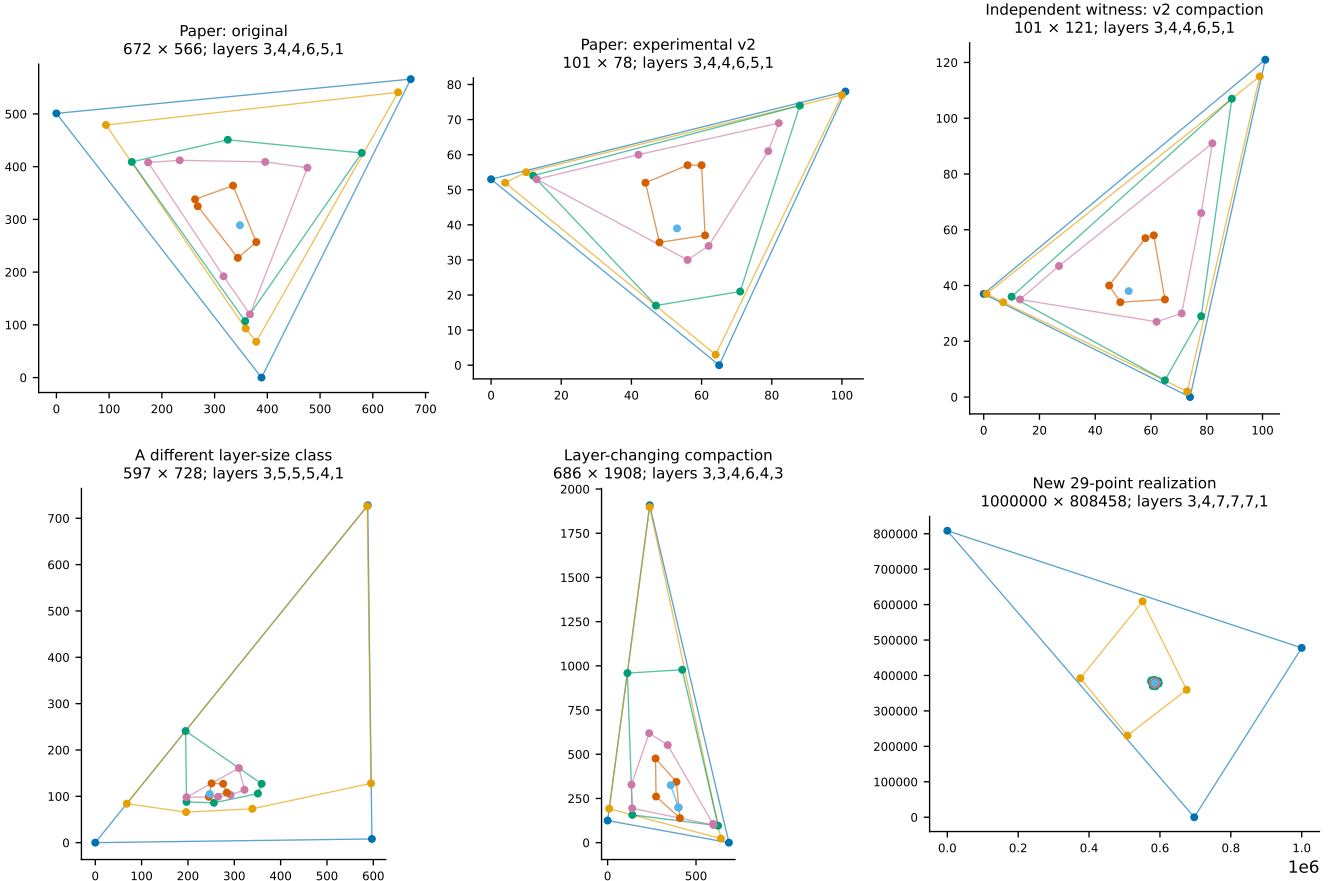


Figure 1: Exact configurations from different sources. Colors show successive convex-hull layers. Panels use independent coordinate scales; box sizes and layer sequences are shown explicitly. Seeded compaction is not fresh discovery.

3.2 Frozen generalization and same-mode ablation

Two additional 23-point order types, two newly found 29-point realizations and known 26/32-point controls were reserved for frozen-v2 tests. The optimizer was not tuned after observing them. Separate comparisons use old versus v2, and frozen v1 versus v2 with identical layer-preservation settings, seeds and budgets. The optimizer README and JSON manifests retain all outcomes, including ties and regressions. The following comparison uses 30 seconds per input and seed 1, with layer-size preservation enabled in both arms.

Input	Standard v1	Experimental v2
heldout23-job090	147×121	177×114
heldout23-job072	432×382	825×905
heldout29-trial11	$5,972 \times 2,295$	$7,352 \times 3,402$
heldout29-trial19	$2,347 \times 7,143$	$5,397 \times 1,151$
control32	35×164	82×84
control26	36×63	46×71

V2 improved area on one of these six inputs and regressed on five. Consequently v1 is the standard CLI default; v2 is explicitly experimental. The better paper result is not sufficient evidence for promoting v2. This cohort was used to select the default, so it is selection evidence, not an untouched estimate of the selected policy’s future performance. Old-versus-v2 cross-family compaction improved all six boxes, but those comparisons used different preservation constraints and do not isolate implementation speed. Across all four compaction phases, 43/43 saved outputs passed independent exact validation, using 3,949.9 CPU seconds.

4 PointSAT and Localizer changes

PointSAT now retains parsed CNFs and incremental SAT checkers per worker, caches exact flippability results, avoids repeated clause shuffling during realization, schedules bounded work, and records stage timings. Optional feedback releases actual orientation assumptions selected from UNSAT cores, solves for a nearby abstract target, and warm-starts the native search. Numerical realization failure alone never produces a permanent SAT exclusion. Exact product-inequality nonrealizability certificates may supply sound cuts only after an independent checker verifies every premise and cancellation.

The self-contained `localizer/` source tree includes the established cached incident-constraint evaluator, early rejection, correct coupled symmetry proposals, archive fixes, warm starts, native budgets, and checked signal saves. Untouched upstream and optimized-v6 baselines, build targets, tests, provenance and upstream attribution are retained. No nested Git repository or modification of a separately cloned upstream checkout is needed. The pinned upstream snapshot contains no explicit license grant; the package documents that fact rather than inventing an upstream license.

4.1 Established speed evidence

Five paired 100,000-iteration tests per family compared cached and reference evaluation within the same revised search, with byte-identical coordinates. Median CPU ratios were 1.879 (23), 2.119 (29), 2.078 (32), and 2.116 (26). These are fixed-work native speedups, not end-to-end solve-rate multipliers. A separate matched eight-input orchestration test used the same unchanged upstream Localizer: wall time fell from 91.945 to 79.791 seconds (13.2%); both versions solved 0/8. Loaded-machine measurements are not bare-metal rates.

4.2 Additional micro-optimizations tested, not promoted

Two new candidates removed a redundant sampling-weight pass and replaced a stop-flag mutex read with an atomic read. Eighty serial runs across all four families compared baseline, either change, and both changes, using five repeats and 300,000 iterations. Every coordinate byte and search counter matched. Neither change consistently reduced CPU time across families; both remain disabled by default. Additional equivalence tests and sanitizers passed. The earlier 831-run prospective cold-search experiment likewise did not support enabling optional line/paired moves universally. A failed optimization experiment is retained as a negative result, not silently presented as an improvement.

Family	v6	Sampling	Atomic	Both
mixed23	1.024	1.015	1.038	1.015
holes29	2.253	2.335	2.256	2.325
gons32	3.277	3.348	3.379	3.393
caps26	2.506	2.460	2.505	2.741

Entries are median CPU seconds for identical fixed work; smaller is better.

5 Matched feedback-on/off experiments

This addresses the previously missing causal comparison. Each arm starts from the same frozen SAT target and native seed. The control uses ordinary warm retries; the treatment uses target-margin core feedback. Both share a 15-second native-stage cap and a 60-second total operational budget including loading, flippability, native search, feedback and acceptance work. All four paper families are represented. Post-hoc independent geometric checks are outside the search budget, identical for both arms, and never feed the search.

The first cohort uses four distinct targets per family and two native seeds: 64 arm trials. A separately registered prospective extension uses the next four targets per family, unchanged code and policy, for another 64. Targets, not repeated seeds, are the independent clusters. SAT generation is excluded because the question is search strategy on matched existing targets. This cannot be interpreted as a complete end-to-end discovery-time benchmark.

Family	Retry valid	Feedback valid	Retry only	Feedback only
mixed23	0/16	0/16	0	0
holes29	1/16	6/16	0	5
gons32	0/16	0/16	0	0
caps26	0/16	0/16	0	0

A pair is one target and native seed. Counting a target as solved if either seed succeeds gives the following cluster-level comparison; only fully evaluated targets are included.

Family	Targets	Retry only	Feedback only	Both	Neither
mixed23	8	0	0	0	8
holes29	8	0	2	1	5
gons32	8	0	0	0	8
caps26	8	0	0	0	8

The recorded 128 arm trials accumulated 7,486.0 parent-observed operational CPU seconds. Post-hoc audits consumed 562.8 summed wall seconds, separately accounted for.

Exit audit: 0 unexpected worker exits, 0 operational inspection errors. 3 trials carry conservative CPU-accounting flags for signal shutdown; budget-triggered signal exits with saved checkpoints are classified as censored searches, not algorithmic failures.

The two first independently certified 29-point outputs illustrate why attribution matters. One was found in the shared initial 15-second stage and is not evidence for either retry policy. Another appeared after three feedback repairs while its matched warm-retry control failed within budget, a paired benefit. Both passed all 475,020 six-subset checks and a fresh extension plus explicit scan of the original 825,978-clause CNF. Their convex-six counts, 4,628 and 4,513, prove the two types differ. Remaining-error counts are search proxies, not certificates.

No general percentage improvement should be inferred from a handful of discordant pairs. The final benchmark reports distinguish shared-initial-stage successes, treatment-only successes, target clustering, CPU use, watchdog censoring, and any errors. A run stopped at its budget is not an UNSAT result.

6 Earlier geometry and the exact 19-point result

The initial direct search implemented Overmars-style insertion, motion and backtracking, fixed-size annealing, conflict-guided repair and exact arrangement relocation. It grew valid 22-point sets from scratch and reached 23 points with one remaining empty hexagon, but did not independently discover a valid 23-point set in that hour. Direct seeded compaction and SAT-seeded repair were useful. The exact relocation scan ruled out all single-point repairs of one particular saved trap, not all 23-point configurations.

The supplied 19-point CNF uses 327 compressed signed C3 orientation orbits, not the ordinary first 969 triple variables. The explicit adapter was independently checked. An initial abstract assignment and one relaxed target were proved nonrealizable by exact product-inequality certificates; those statements did not rule out the geometric problem. Another SAT-guided target was realized.

An exact solution is the origin together with the 120-degree and 240-degree rotations of these six integer representatives:

$$\begin{aligned} &(-667, -1605), (3333, 1586), (-2656, 1313), \\ &(-948, 1232), (2362, 1537), (33666, 3722). \end{aligned}$$

For (a, b) the rotations are $((-a - b\sqrt{3})/2, (a\sqrt{3} - b)/2)$ and $((-a + b\sqrt{3})/2, (-a\sqrt{3} - b)/2)$. Every one of 969 determinants is nonzero. Exhaustive exact checks of 27,132 six-subsets find 28 convex hexagons: 15 with one interior point, 12 with two, and one with four, never zero or three. Independent Python and C++ quadratic-field checks agree, and all 2,311,196 original CNF clauses pass. The final warm realization took 11 seconds; its multi-sample job took 552.032 seconds. These are not the same timing quantity.

The separate symmetry-free arm generated 639 new independently verified impossibility proofs and two unresolved abstract candidates. Direct 19-point searches evaluated billions of proposals but retained forbidden hexagons. Those negative arms do not detract from the separately certified positive solution, and they do not establish a universal ordering of SAT and geometric methods.

7 Validation, limitations, and next work

The workflow/persistence suite includes transactional interruption and SQLite failure injection, exact recovery/deduplication, two-worker delayed archive writes and shared warm references, original-artifact ZIP recovery, settings conflict rejection and retained-result reporting. The concurrency regression passed 30 repeated executions. All 22 workflow tests and 25 legacy pipeline tests passed at the recorded QA snapshot. Native Localizer unit/CLI/equivalence tests and sanitizers passed; standard v1 passed 133 tests and experimental v2 passed 138 tests, including 120 independent full-count comparisons across the four families. Five benchmark tests also passed. The final evidence audit checked all 128 registered trials, 64 target/seed pairs and 2,539 compressed artifact hashes.

The new defaults do not promise a particular grid size or universal solution rate. Compaction benefits depend strongly on the orientation type, preservation mode, coordinate conditioning and budget. Best-so-far outputs are not proven minimum grids. To study layer classes, keep mode and budgets fixed and report all inputs, not only the most compact example. Future work should add more independent targets and predeclared solver-budget allocations, not retune on a single successful configuration.

Potential separate research targets from the earlier investigation include garment-number configurations and balanced colored empty-triangle constructions; their definitions and primary references are recorded in `improvements/RESEARCH_TARGETS.md`. They are proposals, not claimed solved problems in this report.

8 Source and artifact inventory

The repository retains code, immutable benchmark configurations, inputs and binary hashes, exact witnesses, and negative/censored results. Primary entry points and evidence are:

- `docs/USAGE.md`: installation, defaults, recovery and export.
- `pointsat/`: CLI, SQLite storage, exact coordinate utilities and automated optimization.

- `localizer/`: source, baselines, build and micro-optimization ablations.
- `optimizer/README.md`: algorithm, preservation modes, exploratory and held-out results.
- `benchmarks/README.md`: matched strategy protocol and final cohort reports.
- `benchmarks/success29/`: exact 29-point outputs and original-CNF certificates.
- `improvements/success19/`: exact C3 coordinates, diagram and independent certificates.
- `improvements/FINAL_RESULTS.md` and `direct/RESULTS23.md`: earlier experiments.

This PDF is generated by `reports/build_report.py`. Its build manifest records the source inputs and hashes. No search is launched by the report builder.